利用多張GPU訓練大型語言模型：從0開始介紹 DeepSpeed, Flash Attention, Liger Kernel 和 Quantization

Overview

How to train a LLM?

Generally, this PyTorch code will not work...

```
for epoch in range(num_epochs):
for batch in dataloader:
inputs, targets = batch
outputs = model(inputs)
loss = loss_fn(outputs, targets)

optimizer.zero_grad()
loss.backward()
optimizer.step()
```

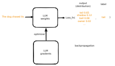
Because LLM is too LARGE to fit in single GPU.

Therefore, we still need packages like:

multi-GPU?	long context?	small vRAM inference?
DeepSpeed	Liger Kernel Flash Attention	Quantization

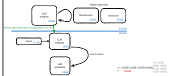
Introduction

1. Forward & Backward pass


output (distribution)
label
The dog chased its
LLM weights
Loss_fn(
tail 0.65
shadow 0.12
ball 0.08
owner 0.02
...
,
tail
)
optimizer
LLM gradients
backpropagation

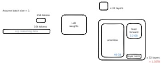
2. Memory needed, assuming LLM is a 8B model

fp16 : 8B params = 8 * 10^9 * 16bit = 16GB


Adam optimizer
LLM weights
32GB
Momentum
32GB
Variance
32GB
Utilize optimized fp16 computation on GPU
32-bits
16-bits
input
?? GB
LLM weights
16GB
loss function
LLM gradients
16GB
T4: 15GB
4090: 24GB
A100: 40GB
H100: 80GB

=> 16GB+16GB+32GB+64GB
= 128GB

3. activations


Assume batch size = 1:
256 tokens
16k tokens
e.g. reasoning data
LLM weights
x 32 layers
attention
40 GB
feed forward
2.2 GB
layer norm
x 32 layers
= 1.35TB

3.1 Activation recomputation (aka gradient checkpointing)

Save "important activations" during forward pass, then recompute when needed in backward pass
(computation overhead)

4. batch size

batch size should be large enough to provide clear gradient.
Generally 4-60M tokens per batch.
(DeepSeek V3 uses batch size 1920 for 32K context, which is 61M tokens)

4.1 Gradient accumulation

Global batch size = mini batch size * gradient accumulation
e.g. 1920 = 16 * 120

5. Challenges

(source: https://huggingface.co/spaces/nanotron/ultrascale-playbook)

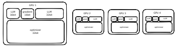
Part I: parameters, gradients and optimizer states
Part II: activations
Part III: quantization

Part I parameters, gradients and optimizer states

1. How to leverage multiple GPUs?

- * We have to compute billions of optimization steps, with large batch size
- * The model may be too large to fit in a single GPU
- * The input can be long. Self-attention takes O(N^2) memory.

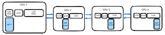
Assume we have 4 GPUs:


GPU 1
LLM 16bit
gradient 16bit
LLM 32bit
optimizer 32bit
GPU 2
LLM
grad
LLM
optimizer
GPU 3
LLM
grad
LLM
optimizer
GPU 4
LLM
grad
LLM
optimizer

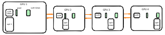
⚠ Most of GPUs cannot fit all the components

2. DeepSpeed - Zero Redundancy Optimizer (ZeRO)

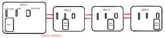
Zero-1


GPU 1
LLM 16bit
grad
LLM 32bit
opti 1
GPU 2
LLM
grad
LLM
GPU 3
LLM
grad
LLM
GPU 4
LLM
grad
LLM

Zero-2


GPU 1
LLM 16bit
grad
LLM 32bit
opti 1
GPU 2
LLM
GPU 3
LLM
GPU 4
LLM

Zero-3


GPU 1
LLM 16bit
grad
LLM 32bit
opti 1
GPU 2
GPU 3
GPU 4
NVLink: 900GB/s

3. Memory Reduction

3.1 Training speed

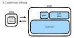
- * Varies based on network bandwidth, model size, and specific hardware configuration.
- * Generally, it will not slow down by an order of magnitude.

4. Zero - Offload

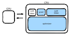
GPU ram
1x
CPU ram
10x

- * Revisit: 16bit world & 32bit world
- * Warn: using offload can be very slow

4.1 optimizer offload


CPU
GPU
LLM 16bit
grad
LLM 32bit
optimizer

4.2 optimizer + model parameter offload


CPU
GPU
LLM 16bit
grad
LLM 32bit
optimizer

5. Simple Experiment on TWCC(國網中心)

https://github.com/anthony-wss/deepspeed-boilerplate/tree/main

4 gpus: with offload

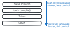
speed: 15GB per gpu, 74s / step (batch_size=1, max_length=512)

8 gpus: no offload

speed: 7.3s / step (batch_size=1, max_length=512)
- vRAM: 24-32GB(varies but will not OOM) per gpu

Part II activations

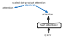
1. kernel


Naive PyTorch
torch.compile()
Triton
CUDA
high-level language
slower, less control
low-level language
faster, full control

2. Flash Attention Algorithm

Faster training & less memory by optimized fetching from CPU RAM

2.1 Speed


scaled dot-product attention
identical
attention
attention
flash attention*
Q K V

(source: Attention Is All You Need)

space: O(N^2)

space: near O(N)

2.1.1 Fused Kernel

(source: https://github.com/Dao-AILab/flash-attention)

2.2 Memory

A = Attention(Q, K, V)
Q
V
A
K
GPU
CPU

2.3 Performance

3. Liger Kernel

Reimplement some core LLM computation with optimized Triton code.

3.1 How to use

```
from transformers import AutoModelForCausalLM
model = AutoModelForCausalLM.from_pretrained("/path/to/model")
```

↓

```
from liger_kernel.transformers import AutoLigerKernelForCausalLM
model = AutoLigerKernelForCausalLM.from_pretrained("path/to/some/model")
```

* Support:
LLaMA, Mistral, Gemma, Paligemma, Qwen, Phi

3.2 Performance

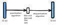
Speed

Memory

(source: https://github.com/linkedin/Liger-Kernel)

Part III Quantization

1. Lossy compression


quantization algorithm
dequantization algorithm
32 bit
8bit int4
4bit int8
2bit
1bit
32 bit

GPTQ
AWQ
GGML family
BitsAndBytes

* Some algorithm just compress a part of LLM for better accuracy

Meta-Llama-3.1-8B-Instruct-Q8_0.gguf → 8B x 8bit / weight = 8GB GPU RAM
(T4 GPU: ~15GB vRAM ✓)

Take away

You may need packages like DeepSpeed, Flash Attention, or Liger Kernel
to train LLM with multiple gpus and limited memory.

Recommended Reading

- https://huggingface.co/spaces/nanotron/ultrascale-playbook
- https://huggingface.co/docs/transformers/deepspeed